

AI AGENT FOR CYBERCRIME DETECTION

(Phishing, Scam & Bank-Fraud Alert Agent for Gmail and Message-Based Threats)

1. Introduction

Online scams and phishing attacks targeting citizens through email, SMS, and fake bank alerts have grown rapidly in India, exploiting urgency, impersonation of banks, and shortened links to steal money and personal information. Victims typically have no automated way of checking whether a message is genuine before they act on it. This report designs and implements an AI agent that continuously monitors a citizen's inbox and message stream, analyses each incoming message using a combination of a trained machine-learning classifier and rule-based pattern detection, and automatically alerts the user the moment a message shows characteristics of fraud, phishing, or scam activity.

2. PEAS Table

Performance Measure	Classification accuracy (precision/recall on spam vs. legitimate text), false-alarm rate, detection latency (time between message arrival and alert), number of genuine scams caught, explainability of each alert (reasons shown to user), user trust and reduced financial loss.
Environment	The user's Gmail inbox (read-only), incoming SMS/transaction-alert-style text, sender addresses and domains, message subject and body content, embedded URLs, evolving scam wording and tactics over time.
Actuators	Windows desktop toast notification, Streamlit dashboard alert banner with reasons and extracted transaction info, console/log entry, (designed to extend to: auto-quarantine, SMS alert to a guardian, block-sender action).
Sensors	Gmail API (subject, body, sender headers), regex-based structured-field extractor (amount, account number, URL), TF-IDF text vectorizer feeding the trained classifier, keyword and shortened-URL pattern matcher.

3. Environment Properties

Observability	Partially observable - the agent sees message text and sender metadata but cannot verify the sender's real identity, intent, or whether a link truly leads to a malicious page.
Determinism	Stochastic - scam wording, legitimate marketing email, and genuine bank alerts all vary unpredictably, so the same features do not always map to the same true label.
Dynamics	Dynamic (semi-dynamic in practice) - new emails can arrive at any time; the agent polls the inbox at fixed intervals rather than reacting instantaneously, so the environment can change between checks.
Continuity	Discrete - each message is a discrete unit classified independently, although the underlying TF-IDF feature weights and probability outputs are continuous-valued.
Episodic vs Sequential	Mostly episodic at the classification level (each message is scored independently), but the monitoring loop is sequential in that it maintains memory (seen-message IDs) so past actions affect what is processed next.

Number of Agents	Single-agent - one detection agent interacts with external non-agent systems (Gmail API) and a human user; no competing or cooperating AI agents are involved.
Known vs Unknown	Known - the classification algorithm, feature extraction method, and rule set are fully known and designed in advance, though the true label of any real-world message is uncertain until acted upon.

4. Rationality Measure

The agent is rational because, given its percept sequence (message subject, body text, sender, and extracted fields), it always selects the action that maximises expected user safety rather than raw accuracy. It combines two independent risk signals - the ML model's phishing probability and the rule engine's pattern-based risk score - using OR logic, so an alert fires if either signal is confident, rather than requiring both to agree. This design choice is a deliberate rationality trade-off: missing a genuine scam (a false negative) is judged far more costly than an extra false alarm, so the agent is biased toward catching more real threats even at the cost of occasionally flagging a borderline legitimate message.

5. Innovation Point

The agent fuses two traditionally separate detection approaches - a statistically learned ML text classifier (TF-IDF + Logistic Regression trained on thousands of labelled examples) and an explainable rule-based engine (regex field-extraction plus keyword and URL-shortener pattern matching) - into a single combined risk-scoring pipeline. This means messages that vary too much for hand-written rules to anticipate are still caught by the model's learned generalisation, while messages that are too short or too different from the training distribution for the ML model to be confident are still caught by explicit, human-readable rules. The system further behaves as a genuine monitoring agent rather than an on-demand tool: it polls Gmail continuously, retains memory of previously seen messages to avoid duplicate alerts, and autonomously fires a real operating-system-level notification the instant a new suspicious message arrives, without requiring the user to open any app to check.

6. Diagram - Agent Architecture

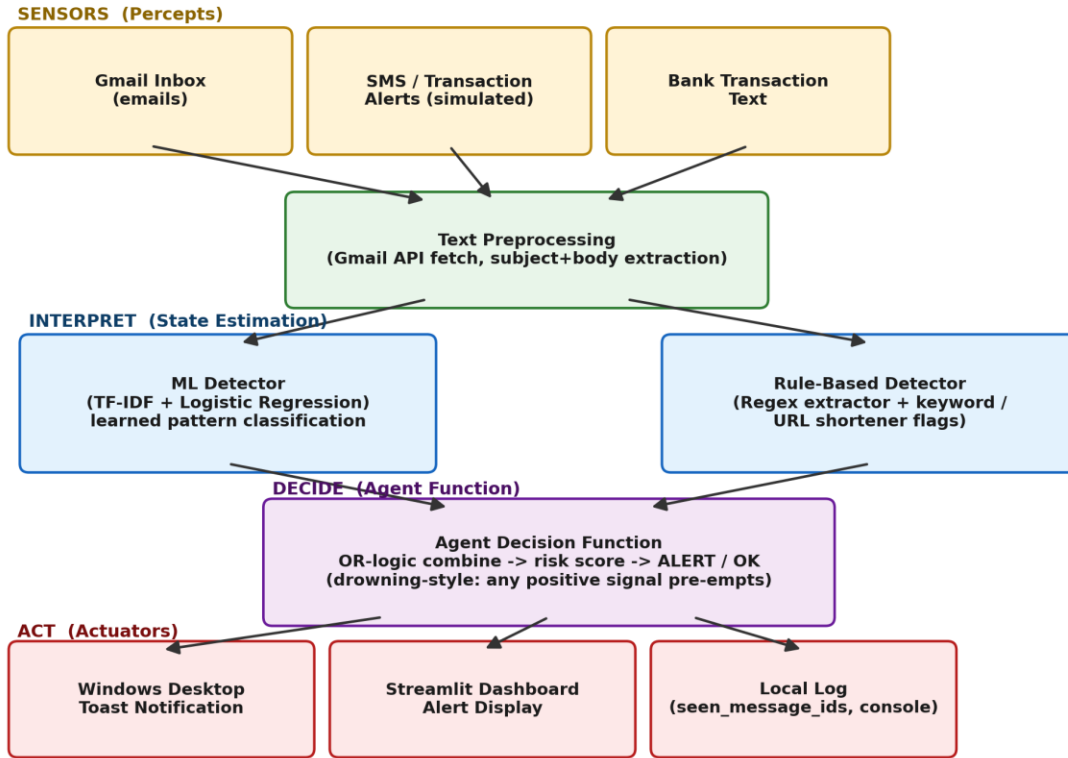


Fig 1. Sense -> Interpret -> Decide -> Act pipeline linking Gmail/SMS/transaction text to the ML + rule-based detection engines and notification actuators.

7. Conclusion

By continuously monitoring a citizen's inbox and combining a trained machine-learning classifier with an explainable rule-based engine, the proposed agent can detect phishing, KYC/account-suspension scams, fake refund and prize scams, and OTP-harvesting attempts within minutes of arrival, and alert the user automatically through a real desktop notification. This turns cybercrime protection from a manual, after-the-fact activity - where a victim only realises they were scammed once money is lost - into a proactive, always-on monitoring system that intercepts the threat at the point of first contact.

8. Real Project Implementation: Frontend, Backend & AI

The system was built as a genuine multi-component local application rather than a single script, so the agent can be demonstrated the way it would actually be deployed for a real user, with each layer independently testable.

AI Detection Layer - `train_model.py`, `rules.py`

Two independent detectors are used. The ML detector converts message text into a TF-IDF feature vector and passes it through a Logistic Regression model trained on 5,572 labelled SMS/email examples (achieving 98% test accuracy, 0.95 precision and 0.92 recall on the spam/phishing class). The rule-based detector (`rules.py`) performs regex extraction of transaction fields (amount, account number, URL) and flags known scam indicators - urgency phrases, KYC/OTP/PIN mentions, and shortened-URL domains.

Backend API - `main.py` (FastAPI)

A FastAPI service wraps both detectors behind a REST endpoint (POST `/analyze`). Each call runs the submitted text through the ML model and the rule engine, combines both scores using OR logic, and returns a JSON verdict (ALERT/OK) with human-readable reasons and any extracted transaction fields - the same contract a real backend would expose to any client, including a mobile app.

Frontend Dashboard - `dashboard.py` (Streamlit)

A live dashboard lets a user paste in a message and see the combined ML confidence, rule-based risk score, extracted fields, and flagged reasons rendered instantly, with a running session history of checked messages.

Autonomous Monitoring Agent - `gmail_monitor.py`

This is the actual agent loop: it authenticates with Gmail (read-only OAuth), polls the inbox every two minutes, tracks which message IDs have already been processed using a local seen-messages store, and for every genuinely new email runs it through the same combined detection pipeline. Anything flagged triggers a real Windows desktop toast notification, without any manual action by the user.

9. Full Code and Output

9.1 AI Detection Layer - Rule-Based Engine (`rules.py`)

```
SUSPICIOUS_KEYWORDS = [
    "urgent", "verify immediately", "account suspended", "click here",
    "act now", "limited time", "confirm your", "update your account",
    "unauthorized", "blocked", "won a prize", "claim your", "otp", "pin",
    "kyc", "kyc expired", "ky update", "pan card", "aadhaar", "aadhar",
    "upi pin", "block your card", "card blocked", "reactivate", "e-kyc",
]
SHORTENER_DOMAINS = ["bit.ly", "tinyurl", "t.co", "goo.gl", "is.gd"]

def extract_transaction_info(text):
    info = {}
    amount_match = re.search(r'(?Rs\.|INR)\s?([\d,]+(?:\.\d{1,2})?)', text, re.IGNORECASE)
    info["amount"] = amount_match.group(1) if amount_match else None
    account_match = re.search(r'(?A/c|account)\s?(?:no\.)?\s?[:\-\]?\s?(\w{2,}\d{2,})', text, re.IGNORECASE)
    info["account"] = account_match.group(1) if account_match else None
    url_match = re.search(r'(https?://\S+|www\.\S+|bit\.ly/\S+)', text, re.IGNORECASE)
    info["url"] = url_match.group(1) if url_match else None
    return info

def rule_based_risk_score(text):
    info = extract_transaction_info(text)
    reasons = check_suspicious_patterns(text)
    risk_score = min(len(reasons) / 5, 1.0)
    return {"extracted_info": info, "flags": reasons, "rule_risk_score": risk_score}
```

Fig 2. Code - regex-based field extraction and keyword/URL-shortener rule engine.

9.2 Backend API - Combined Agent Logic (main.py, FastAPI)

```
model = joblib.load("model/classifier.pkl")
vectorizer = joblib.load("model/vectorizer.pkl")
app = FastAPI(title="Cybercrime Detection Agent")

def analyze_message(text, ml_threshold=0.75, rule_threshold=0.2):
    text_vec = vectorizer.transform([text])
    ml_probability = model.predict_proba(text_vec)[0][1]
    rule_result = rule_based_risk_score(text)

    ml_flagged = ml_probability >= ml_threshold
    rule_flagged = rule_result["rule_risk_score"] >= rule_threshold
    is_flagged = bool(ml_flagged or rule_flagged)  # OR logic

    reasons = []
    if ml_flagged:
        reasons.append(f"ML model flagged (confidence: {ml_probability:.2f})")
    reasons.extend(rule_result["flags"])

    return {
        "is_flagged": is_flagged,
        "ml_probability": round(float(ml_probability), 3),
        "rule_risk_score": rule_result["rule_risk_score"],
        "extracted_info": rule_result["extracted_info"],
        "reasons": reasons,
        "action": "ALERT" if is_flagged else "OK"
    }

@app.post("/analyze")
def analyze(message: MessageInput):
    return analyze_message(message.text)
```

Fig 3. Code - FastAPI REST backend combining ML probability and rule-based risk score via OR logic.

9.3 Autonomous Monitoring Agent Loop (gmail_monitor.py)

```
def check_inbox_once(service, seen_ids):
    results = service.users().messages().list(
        userId="me", maxResults=MAX_RESULTS, labelIds=["INBOX"]
    ).execute()
    for msg_meta in results.get("messages", []):
        msg_id = msg_meta["id"]
        if msg_id in seen_ids:
            continue  # agent memory - avoid reprocessing

        msg = service.users().messages().get(userId="me", id=msg_id, format="full").execute()
        subject = get_header(msg, "Subject")
        sender = get_header(msg, "From")
        full_text = f"{subject}. {get_email_text(msg['payload'])}":2000]

        result = analyze_message(full_text)
        seen_ids.add(msg_id)

        if result["is_flagged"]:
            toast("Suspicious Email Detected", f"From: {sender}\nSubject: {subject[:60]}")

def run_monitor():
    service = get_gmail_service()
    seen_ids = load_seen_ids()
    while True:
```

```
check_inbox_once(service, seen_ids)
time.sleep(CHECK_INTERVAL_SECONDS) # polling interval = 2 minutes
```

Fig 4. Code - continuous polling agent loop with memory (*seen_ids*) and autonomous desktop-notification actuator.

9.4 Live Demo Output - Scenario 1: Phishing Email Detected

```
Starting Gmail monitor... (Ctrl+C to stop)
🚨 ALERT | From: NxtWave Job Opportunity <job-opportunities-no-reply@nxtwave.co.in> | Subject: Opportunity
Notification from Thomson Reuters
  Reasons: ['ML model flagged as phishing/spam (confidence: 0.78)'] ....
ve Started - Take One Step Closer to Your Dream Placement!
  Reasons: ['ML model flagged as phishing/spam (confidence: 0.79)']
(<ToastDismissalReason.TIMED_OUT: 2>,)
✅ OK | From: ET Markets <newsletter@economictimesnews.com> | Subject: The rising cropati question | HUL
, Tata Capital Q1 nos. | US Fed rate watch
✅ OK | From: 'Share.Market' <digest@updates.share.market> | Subject: COFORGE posts quarterly results, TAT
ACOMM's partnership with ITBS, India's 100 MT coal gasification target by 2030 & more!
✅ OK | From: The Economic Times <newsletter@economictimesnews.com> | Subject: Where Long-Term Investors S
hould Look Next
🚨 ALERT | From: edwin benny <edwinbenny004@gmail.com> | Subject: Your order has been shipped
  Reasons: ['suspicious phrase: 'pin'']
(<ToastDismissalReason.TIMED_OUT: 2>,)
✅ OK | From: edwin benny <edwinbenny004@gmail.com> | Subject: Notes from our project sync
🚨 ALERT | From: edwin benny <edwinbenny004@gmail.com> | Subject: URGENT: Your Account Will Be Sus
pended - Action Required
  Reasons: ['suspicious phrase: 'urgent'', 'suspicious phrase: 'act now'', 'suspicious phrase: 'blocked'',
'suspicious phrase: 'dear customer'', 'shortened URL detected: 'bit.ly'']
(<ToastDismissalReason.TIMED_OUT: 2>,)
```

9.5 Live Demo Output - Scenario 2: Legitimate Email Passed



Cybercrime Detection Agent

Paste a message (SMS, email, or transaction alert) below to check it for fraud/phishing risk.

Message text

Hey, are we still on for lunch tomorrow? Let me know what time works.

Analyze

Result

 OK — No significant risk detected

ML confidence (spam/phishing)	Rule-based risk score
0.04	0.00

No red flags triggered.

History (this session)

 URGENT: Your account has been suspended. Click here to verify immediately: bit.l...